

COLLEGE FOOTBALL FANTASY

Security Architecture

DSHub Build2Earn — Cohort C, Team 4
Cybersecurity Engineering Deliverable

Document Version	1.0
Date	September 2026
Project	College Football Fantasy
Primary Focus	Application, API, data and platform security

1. Purpose

This document defines the practical security architecture required for the College Football Fantasy MVP. It focuses on the platform components that directly handle user accounts, fantasy teams, player data, gameweeks, scoring, leagues, rankings and administration.

The architecture is scoped to the project and prioritizes protection of fantasy results, user accounts, application data and administrative functions.

2. Security Priorities

- Account security — prevent unauthorized account access.
- Access control — users access only their own teams and permitted league resources.
- Fantasy integrity — prevent manipulation of squad rules, salary limits, scoring and rankings.
- API security — authenticate, authorize and validate protected requests.
- Data protection — protect user, team and player information.
- Administrative security — restrict sensitive platform operations.
- Monitoring — record and detect important security events.

3. Security Architecture Overview

Core principle: the frontend is an untrusted client. The backend is the authoritative security and business-logic layer.

Component	Security responsibility
Web / Mobile Frontend	Display data and collect input. Client-side validation is for user experience, not security.
REST API / Backend	Authentication, authorization, validation, business rules and access control.
Fantasy Team / Squad Logic	Enforce 16-player squad, position requirements and \$100M salary cap.
Scoring / Ranking Logic	Calculate points and rankings server-side.
PostgreSQL	Store application data with restricted access and protected credentials.
College Football Data API	External source; validate and normalize incoming player/performance data.
Admin Functions	Restricted by role and protected with audit logging.
Logging / Monitoring	Record important authentication, authorization, admin and integrity events.

4. Authentication

4.1 User Authentication

- Users authenticate before accessing protected account and fantasy-team functions.
- Passwords are securely hashed and never stored in plaintext.
- Email verification should be supported.
- Password recovery uses a controlled, time-limited process.
- Login attempts are protected against brute-force and automated abuse.
- Authentication sessions/tokens use HTTPS/TLS.

4.2 Administrative Authentication

- Administrative accounts require stronger protection than standard users.
- MFA should be used for privileged access where supported.
- Administrative sessions require appropriate expiration and access controls.

5. Authorization and RBAC

Authorization is enforced by the backend after authentication. The MVP uses a simple role model that can expand later.

Role	Required permissions
Standard User	Manage own profile, fantasy team, lineup, captain and permitted league activity; view player data and rankings.
League Administrator	Manage permitted league functions for leagues they administer.
System Administrator	Manage approved platform operations such as player data, gameweeks and authorized corrections.
Privileged Administrator	Perform highly sensitive security/platform operations when specifically required.

Resource ownership rule: authentication alone does not grant access to another user's team. The backend verifies ownership or explicit permission for every protected resource.

6. API Security

- Use HTTPS/TLS for all client-to-API and service communication.
- Protect private endpoints with authentication.
- Perform authorization checks on every protected operation.
- Validate request bodies, parameters, IDs, types and allowed values server-side.
- Apply rate limits to authentication and abuse-sensitive endpoints.
- Do not expose passwords, tokens, API keys or unnecessary sensitive information.
- Use safe error messages.
- Use parameterized database queries/prepared statements.
- Keep API credentials and signing secrets outside source code.

7. Fantasy Team and Squad Security

Fantasy-team integrity is a core security requirement because users must not bypass game rules through modified requests.

- Enforce the 16-player squad: 11 starters and 5 substitutes.
- Enforce the \$100M salary cap on the server.
- Validate player eligibility and position requirements server-side.
- Validate transfers and substitutions against approved rules.
- Reject lineup changes after the applicable gameweek/player lock deadline.
- Allow users to modify only teams they own or are authorized to manage.

8. Player Data and Scoring Integrity

Player and performance information from the external data source must be validated before it affects fantasy results.

- Validate incoming player and performance records.
- Use stable player identifiers and prevent duplicate/conflicting records.
- Do not allow ordinary users to modify official statistics, prices or availability.
- Calculate fantasy points on the backend using approved scoring rules.
- Users cannot submit or directly change their fantasy points.
- Generate rankings from controlled scoring data.
- Restrict and audit authorized score corrections.

Secure scoring flow: External Player Data → Validation → Normalization → Scoring Engine → Team Points → Ranking Service.

9. Database Security

- Restrict PostgreSQL access to authorized application services and administrators.
- Use dedicated database credentials.
- Protect credentials using secrets management.
- Use encryption in transit and appropriate encryption at rest.
- Apply least-privilege database permissions.
- Use parameterized queries/prepared statements.
- Maintain backups and recovery procedures.
- Never expose database credentials or internal database details to the frontend.

10. Secure Handling of Project Data

Data	Required protection
User account data	Authentication, authorization and controlled API exposure.
Fantasy teams	Ownership checks and server-side validation.
Player data	Validation, controlled updates and integrity checks.
Performance/statistics	Validated source data and controlled scoring use.
Scores/rankings	Server-side calculation and restricted modification.
League data	Access control based on membership/league permissions.
Admin records	Restricted access and audit logging.

11. Frontend–Backend Security Boundary

The frontend provides user interaction and presentation but must not be trusted to enforce security-critical rules.

- Frontend validation is for usability only.
- Backend validation is authoritative.
- The frontend must not contain database credentials, private API keys or server signing secrets.
- The frontend must not calculate authoritative scores or rankings.

- The backend re-checks salary cap, squad composition, ownership, permissions and gameweek deadlines.

12. Secrets Management

- Store database credentials, authentication/signing secrets and external API keys in protected environment configuration or an approved secrets-management service.
- Never commit secrets to GitHub or include them in frontend code.
- Use separate secrets for development, testing, staging and production.
- Restrict secret access using least privilege.
- Rotate exposed or compromised credentials.
- Prevent secrets from appearing in logs.

13. Logging and Monitoring

Event	Required handling
Failed/suspicious login	Record security event and monitor repeated abuse.
Authorization failure	Record denied access to protected resources.
Role/permission change	Create an audit record.
Player-data change	Record who/what/when changed data.
Score correction	Record administrator, reason and affected data.
Gameweek configuration change	Audit the change.
Unusual API activity	Monitor for rate abuse or suspicious patterns.

Never log passwords, authentication tokens, API keys or database credentials.

14. Security Boundaries

- User → Frontend: user-controlled environment; all client input is untrusted.
- Frontend → API: HTTPS/TLS, authentication, authorization, validation and rate limiting.
- API → Database: restricted service access, least privilege and protected credentials.
- API → External Data Provider: secure credentials plus validation of returned data.
- User → Admin Functions: separated by RBAC and privileged access controls.

15. Development Security Requirements

- Implement authentication and authorization before protected APIs are exposed.
- Implement backend ownership checks for teams and leagues.
- Implement server-side salary-cap and squad validation.
- Keep scoring and ranking calculations on the server.
- Protect external API and database credentials.
- Add audit logging for administrative and scoring-sensitive actions.
- Test authentication, authorization, input validation and access-control boundaries before production.
- Review dependencies and configuration for known security weaknesses before deployment.

16. Security Acceptance Criteria

ID	Acceptance criterion
SEC-01	Unauthenticated users cannot access protected account/team functions.
SEC-02	Users cannot modify another user's fantasy team.
SEC-03	Backend enforces the 16-player squad and \$100M salary-cap rules.
SEC-04	Users cannot directly modify official player data, points or rankings.
SEC-05	Gameweek/lineup deadlines are enforced by the backend.
SEC-06	Administrative functions require the appropriate role.
SEC-07	Secrets are not stored in source code or frontend bundles.
SEC-08	Security-sensitive administrative and scoring actions are logged.
SEC-09	Database access is restricted and uses protected credentials.
SEC-10	Protected communications use HTTPS/TLS.

17. Scope and Implementation Note

This document is a security design baseline for the College Football Fantasy project. It defines what the implementation should enforce; it does not claim that these controls have already been implemented.

The exact authentication library, session/token approach, cloud secrets service, monitoring platform and database deployment configuration should be finalized by engineering and recorded in the TRD.

18. Conclusion

The architecture concentrates on the platform's core trust points: user accounts, team ownership, salary-cap and squad rules, player/performance data, scoring, rankings, APIs, databases and administrative operations.

Central rule: the frontend displays and requests; the backend authenticates, authorizes, validates, calculates and enforces.